Learning Hub Master Series • Technical Handbook

PYTHON PROGRAMMING FOR LEARNING HUB

คู่มือการเขียนโปรแกรมฉบับวิศวกรรมและการอ้างอิงระดับมืออาชีพ

การสังเคราะห์เนื้อหาหลักสูตรระดับมหาวิทยาลัยอย่างเข้มข้นและครบวงจร (การกำหนดค่า, ชนิดข้อมูล, สคริปต์, มอดูล, คำสั่งเงื่อนไข, ฟังก์ชัน, การวนซ้ำด้วย Range) ผสานกับหัวข้อขั้นสูงเชิงสถาปัตยกรรมระดับมืออาชีพ: Comprehensions, Closures, Decorators, สถาปัตยกรรมเชิงวัตถุ (OOP), Magic Dunder Methods, ลำดับชั้นข้อยกเว้น (Exception Hierarchies), Context Managers และกลไกหน่วยความจำภายใน

- ไดนามิกไทป์ (Dynamic Typing)
- ลำดับข้อมูล & การสไลซ์ (Sequences & Slicing)
- คอมพรีเฮนชัน (Comprehensions)
- First-Class Functions
- เดคอเรเตอร์ (Decorators & Wrappers)
- OOP & โปรโตคอล Dunder
- Context Managers
- สำนวนไพทอนิค (Pythonic Idioms)

ฉบับสังเคราะห์หลักสูตรวิชาการและตำรามาตรฐานสากล (Academic Syllabus & Textbook Synthesis Edition)
แหล่งอ้างอิงหลักสูตร: *University Lecture Slide Series* & *Python Notes for Professionals (GoalKicker)*
มาตรฐานรองรับ: Python 3.8 - 3.12+ (Modern Idiomatic Python) • จัดทำขึ้นสำหรับ CLI_WS Learning Hub

สารบัญ (Table of Contents)

ส่วนที่ 1: รากฐานระบบ, ชนิดข้อมูล & การควบคุมลำดับการทำงาน (Core Foundations, Types & Flow of Control)

บทที่ 1: สภาพแวดล้อมการทำงานของ Python, Interactive Shell & การรันสคริปต์ — Bytecode & PVM, REPL เทียบกับสคริปต์, Stream I/O, F-strings ยุคใหม่, มาตรฐาน PEP 8

บทที่ 2: ตัวแปร, ไดนามิกไทป์ & ข้อมูลตัวเลขพื้นฐาน — การผูกโยงข้อมูลอ้างอิง, int/float/complex/bool, การหารจริงเทียบกับหารปัดเศษ, การแปลงชนิดข้อมูล

บทที่ 3: คำสั่งแยกสาขาเงื่อนไข & ความหมายเชิงตรรกะของบูลีน — Truthiness & Falsiness, การเท่ากัน (==) เทียบกับเอกลักษณ์ (is), Short-circuiting, Match-Case

บทที่ 4: การวนซ้ำ, ลำดับข้อมูล & ฟังก์ชัน range() — For loops, ช่วงครึ่งเปิด range, While loops, โครงสร้าง Loop Else, Enumerate & Zip

ส่วนที่ 2: โครงสร้างข้อมูล & การจัดการข้อความ (Data Structures & Text Processing)

บทที่ 5: สตริง & กลไกการจัดการข้อความ — ความไม่เปลี่ยนรูป (Immutability), การสไลซ์แบบขยาย [::], strip, split, join, การค้นหาข้อความ

บทที่ 6: ลิสต์, ทูเพิล & การสไลซ์ลำดับข้อมูล — การปรับขนาดอัตโนมัติ, การปรับเปลี่ยนข้อมูลในลิสต์, ทูเพิลที่ไม่เปลี่ยนรูป, Unpacking (*args), Shallow vs Deep Copy

บทที่ 7: ดิกชันนารี, เซต & คอลเลกชันแบบแฮช — การสืบค้นความเร็ว O(1), keys/values/items, get พร้อมค่าเริ่มต้น, พีชคณิตเซต (|, &, -, ^)

บทที่ 8: คอมพรีเฮนชัน & นิพจน์เจเนอเรเตอร์ — List/Dict/Set Comprehensions, การกรองเงื่อนไข, เจเนอเรเตอร์แบบประหยัดหน่วยความจำ

ส่วนที่ 3: ฟังก์ชันแบบโมดูลาร์ & การโปรแกรมเชิงฟังก์ชัน (Modular Functions & Functional Programming)

บทที่ 9: กลไกของฟังก์ชัน, พารามิเตอร์ & สโคป (LEGB) — Positional เทียบกับ Keyword, *args/**kwargs, กฎขอบเขต LEGB, กับดัก Mutable Default Argument

บทที่ 10: การโปรแกรมเชิงฟังก์ชัน: แรมบ์ดา, ฟังก์ชันอันดับสูง & อิตเทอเรเตอร์ — First-Class Functions, แรมบ์ดา, map/filter/sorted, โพรโทคอลอิตเทอเรเตอร์, yield generators

บทที่ 11: เดคอเรเตอร์, โคลเชอร์ & แรปเปอร์ฟังก์ชัน — Lexical Closures, ไวยากรณ์ @decorator, functools.wraps, เดคอเรเตอร์วัดเวลาและตรวจสอบความถูกต้อง

บทที่ 12: มอดูล, แพ็กเกจ & สถาปัตยกรรมโครงงาน — กลไกการ import, sys.path, โครงสร้างแพ็กเกจ & __init__.py, __name__ == __main__, มอดูล math

ส่วนที่ 4: การโปรแกรมเชิงวัตถุ (OOP) & โพรโทคอล (Object-Oriented Programming & Protocols)

บทที่ 13: หลักการเชิงวัตถุ: คลาส, แอตทริบิวต์ & เมธอด — __init__ & self, แอตทริบิวต์ระดับอินสแตนซ์เทียบกับระดับคลาส, @classmethod, @staticmethod, @property

บทที่ 14: การสืบทอด, ภาวะหลายรูป & Dunder Methods พิเศษ — การสืบทอด, super(), MRO, Duck Typing, การโอเวอร์โหลดตัวดำเนินการ (__str__, __len__, __add__)

ส่วนที่ 5: สถาปัตยกรรมระบบขั้นสูง: ข้อยกเว้น, สตรีมไฟล์ & การแปลงข้อมูล (Robust Systems)

บทที่ 15: การจัดการข้อยกเว้น & การออกแบบเพื่อความทนทาน — try-except-else-finally, ลำดับชั้นข้อยกเว้น, การ raise ข้อยกเว้น, คลาสข้อผิดพลาดที่กำหนดเอง

บทที่ 16: การจัดการไฟล์, Context Managers & การแปลงรูปแบบข้อมูล — โหมดการเปิด open(), with statement, การแปลง JSON & CSV, pathlib

บทที่ 17: สำนวนไพทอนิค, การเพิ่มประสิทธิภาพ & การป้องกันข้อผิดพลาด — The Zen of Python, กับดักการวนซ้ำ, การจัดการหน่วยความจำ, ภาพรวม GIL, venv & pip

ส่วนที่ 6: ภาคผนวกอ้างอิง (Reference Appendices)

ภาคผนวก A: ตารางลำดับความสำคัญของตัวดำเนินการในภาษา Python (Complete Operator Precedence Table)

ภาคผนวก B: ตารางอ้างอิงฟังก์ชันบิลต์อินมาตรฐานของ Python (Master Built-in Functions Reference Table)

ภาคผนวก C: สรุปการใช้งานเมธอดของสตริงและคอลเลกชัน (String & Collection Methods Master Cheat Sheet)

บทที่ 1

สภาพแวดล้อมการทำงานของ Python, Interactive Shell & การรันสคริปต์

1.1 กระบวนการประมวลผลของ Python: จากซอร์สโค้ดสู่ไบต์โค้ดและ PVM

โดยทั่วไปภาษา Python มักถูกอธิบายว่าเป็นภาษาแบบอินเทอร์พรีเตอร์ (Interpreted Language) ทว่าในสภาพแวดล้อมการทำงานจริงยุคปัจจุบัน (เช่น CPython ซึ่งเป็นมาตรฐานหลัก) สถาปัตยกรรมภายในจะใช้การผสมผสานระหว่างการคอมไพล์และการอินเทอร์พรีต (Hybrid Compilation-Interpretation Architecture) เมื่อนักพัฒนาสั่งรันสคริปต์ภาษา Python (`program.py`) ตัวประมวลผลจะทำงานผ่านสองขั้นตอนหลักอย่างเป็นระบบ:

1. **การคอมไพล์เป็นไบต์โค้ด (Bytecode Compilation)**: ตัวแจงส่วนไวยากรณ์ (Parser) ของ Python จะแปลงซอร์สโค้ดที่มนุษย์อ่านได้ให้อยู่ในรูปแบบคำสั่งขั้นกลางที่ไม่ขึ้นกับแพลตฟอร์มฮาร์ดแวร์ เรียกว่า **ไบต์โค้ด (Bytecode)** โดยชุดคำสั่งไบต์โค้ดที่คอมไพล์แล้วนี้จะถูกบันทึกแคชไว้ในไฟล์นามสกุล `.pyc` ภายใต้ไดเรกทอรี `__pycache__` เพื่อเพิ่มความเร็วในการเริ่มต้นทำงานในครั้งถัดไป
2. **การประมวลผลด้วยเครื่องเสมือน (PVM Execution)**: **Python Virtual Machine (PVM)** ซึ่งเป็นลูปประมวลผลหลักขนาดใหญ่ที่เขียนด้วยภาษา C จะอ่านสตรีมคำสั่งไบต์โค้ดทีละลำดับ และแปลงแต่ละรหัสคำสั่ง (Opcode) ให้กลายเป็นชุดคำสั่งระดับเครื่องจักร (CPU Machine Instructions) ที่สอดคล้องกับสถาปัตยกรรมฮาร์ดแวร์ของเครื่องโฮสต์นั้น ๆ

คุณลักษณะ	โหมดโต้ตอบทันที (Interactive REPL)	โหมดการรันไฟล์สคริปต์ (Script Execution)
การเรียกใช้งาน (Invocation)	พิมพ์คำสั่ง `python3` โดยตรงบนหน้าต่างเชลล์ของระบบปฏิบัติการ	รันคำสั่ง `python3 script_name.py` ผ่านบรรทัดคำสั่ง (CLI)
จุดประสงค์หลัก (Primary Purpose)	การทดสอบแนวคิดอย่างรวดเร็ว (Prototyping), การคำนวณทางคณิตศาสตร์, การสำรวจ API ของมอดูล	การพัฒนาแอปพลิเคชันซอฟต์แวร์ฉบับสมบูรณ์ที่สามารถนำกลับมาทำงานซ้ำได้อย่างคงทน
พฤติกรรมการแสดงผล (Output Behavior)	ประเมินค่านิพจน์และพิมพ์ผลลัพธ์ออกทางหน้าจอโดยอัตโนมัติทันที	จำเป็นต้องใช้คำสั่ง `print()` อย่างชัดเจนเพื่อส่งข้อมูลออกสู่เทอร์มินัล

1.2 สตรีม I/O มาตรฐาน: print() และ input()

การโต้ตอบกับผู้ใช้ผ่านหน้าจอเทอร์มินัลมาตรฐานถูกขับเคลื่อนด้วยสองฟังก์ชันบิลต์อินพื้นฐาน:

- `print(*objects, sep=' ', end='\n', file=sys.stdout, flush=False)` : ทำหน้าที่แปลงข้อมูล (Serialize) ให้เป็นข้อความเพื่อแสดงผล โดยสามารถกำหนดอักขระคั่นระหว่างวัตถุได้ผ่านพารามิเตอร์ `sep` และกำหนดอักขระปิดท้ายบรรทัดได้ผ่านพารามิเตอร์ `end`
- `input(prompt='')` : ทำหน้าที่ระงับการทำงานชั่วคราวเพื่อรอรับข้อความจากผู้ใช้ พร้อมแสดงข้อความนำทาง (Prompt) หากมีการระบุ **ประเด็นสำคัญยิ่งคือ `input()` จะคืนค่ากลับมาเป็นสตริง (`str`) เสมอ** ดังนั้นการคำนวณทางตัวเลขจึงจำเป็นต้องแปลงชนิดข้อมูล (Type Casting) อย่างชัดเจน

1.3 วิวัฒนาการของการจัดรูปแบบสตริง: จาก % สู่ F-Strings ยุคใหม่

ภาษา Python ได้พัฒนาแบบแผนการจัดรูปแบบสตริงผ่าน 3 ยุคสำคัญ:

1. **การจัดรูปแบบสไตล์ printf ของภาษา C (`%`)**: เช่น `"Score: %d, Rate: %.2f" % (score, rate)` (แบบแผนดั้งเดิม)
2. **เมธอด str.format()**: เช่น `"Score: {}, Rate: {:.2f}".format(score, rate)` (เปิดตัวใน Python 2.7 / 3.0+)
3. **Formatted String Literals (F-Strings)**: นำเสนอใน Python 3.6 (PEP 498) โดยนิพจน์ภายในวงเล็บปีกกา `{expr}` จะถูกประเมินค่าโดยตรงขณะรันไทม์ ซึ่งมอบประสิทธิภาพความเร็วสูงสุดและความกระจ่างชัดในการอ่านโค้ด

```python
# ตัวอย่างการใช้งานจริง: การคำนวณอุณหภูมิความรู้สึกหนาวจากแรงลม (Windchill Temperature)
def calculate_windchill(temp_f: float, wind_mph: float) -> float:
    """คำนวณดัชนีความหนาวเย็นตามสูตรมาตรฐานของ National Weather Service"""
    return 35.74 + (0.6215 * temp_f) - (35.75 * (wind_mph ** 0.16)) + (0.4275 * temp_f *
(wind_mph ** 0.16))

temperature = 32.0
wind_speed = 20.0
chill = calculate_windchill(temperature, wind_speed)

# การใช้ f-string ยุคใหม่พร้อมกำหนดทศนิยม (.1f และ .2f)
print(f"Air Temp: {temperature:.1f}°F | Wind: {wind_speed:.1f} mph | Windchill: {chill:.2f}
°F")
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Air Temp: 32.0°F | Wind: 20.0 mph | Windchill: 23.86°F
```

1.4 แบบแผนการเขียนโค้ดและมาตรฐาน PEP 8

Python กำหนดขอบเขตของบล็อกโค้ดผ่าน **การเยื้องบรรทัดอย่างเคร่งครัด (Strict Indentation)** (โดยมาตรฐานกำหนดให้ใช้การเคาะเว้นวรรค 4 ช่องต่อหนึ่งระดับการซ้อน และต้องไม่ผสมปุ่ม Tab กับช่องว่างเด็ดขาด) โดยมีเอกสารข้อกำหนดมาตรฐานที่เป็นที่ยอมรับในระดับสากลคือ **PEP 8**:

- ใช้ช่องว่าง 4 เคาะ (Spaces) สำหรับการเยื้องบรรทัดในแต่ละระดับ
- การตั้งชื่อตัวแปรและฟังก์ชัน: ใช้รูปแบบ `snake_case` (เช่น `calculate_total_price`)
- การตั้งชื่อคลาส: ใช้รูปแบบ `PascalCase` หรือ `CapWords` (เช่น `StudentRecord`)
- ค่าคงที่ระดับมอดูล: ใช้รูปแบบตัวพิมพ์ใหญ่ทั้งหมด `ALL_CAPS` (เช่น `MAX_BUFFER_SIZE = 1024`)
- จำกัดความยาวโค้ดไม่เกิน 79 ตัวอักษรต่อบรรทัด และไม่เกิน 72 ตัวอักษรสำหรับ Docstring

บทที่ 2

ตัวแปร, ไดนามิกไทป์ & ข้อมูลตัวเลขพื้นฐาน

2.1 ตัวแปรในฐานะพอยน์เตอร์อ้างอิง (Name Binding)

ในภาษาโปรแกรมแบบสถิต เช่น ภาษา C ตัวแปรคือตำแหน่งบนหน่วยความจำที่มีการระบุชนิดข้อมูลระดับฮาร์ดแวร์อย่างตายตัว แต่ในภาษา Python **ตัวแปรคือชื่อเชิงสัญลักษณ์ (References หรือ Tags) ที่ถูกผูกโยงเข้ากับวัตถุ (Objects) ซึ่งจัดสรรอยู่บนหน่วยความจำฮีป (Heap Memory)**

เมื่อเราเขียนคำสั่ง `x = 42`:

1. Python จะสร้างอินสแตนซ์ของวัตถุจำนวนเต็มที่มีค่า `42` ขึ้นบนหน่วยความจำฮีป
2. ชื่อตัวแปร `x` จะถูกผูกโยงเข้ากับแอดเดรสหน่วยความจำของวัตถุดังกล่าว
3. เมื่อสั่ง `y = x` ระบบจะไม่ได้ทำสำเนาวัตถุขึ้นมาใหม่ แต่จะผูกโยงให้ `y` อ้างอิงไปยัง **วัตถุตัวเดียวกันทุกประการ**

```
a = [1, 2, 3]
b = a          # b อ้างอิงไปยังอ็อบเจกต์ลิสต์ตัวเดียวกันกับ a
b.append(4)    # การเปลี่ยนแปลงค่าผ่าน b ย่อมส่งผลกระทบต่อ a ด้วย!
print(f"a = {a} | id(a) == id(b): {id(a) == id(b)}")
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
a = [1, 2, 3, 4] | id(a) == id(b): True
```

2.2 ข้อมูลพื้นฐานและจำนวนเต็มความละเอียดไม่จำกัด

ภาษา Python มีชนิดข้อมูลตัวเลขและตรรกะพื้นฐาน 4 ชนิด:

ชนิดข้อมูล	ความเปลี่ยนแปลงได้ (Mutability)	คุณลักษณะด้านหน่วยความจำและความแม่นยำ	ตัวอย่างลิเทอรัล
`int`	ไม่เปลี่ยนแปลงรูป (Immutable)	**ความแม่นยำไม่จำกัด (Arbitrary Precision)** (ถูกจำกัดเพียงขนาดแรมของเครื่องโฮสต์ หมดปัญหาโอเวอร์โฟลว์ 32 หรือ 64 บิต)	`42`, `-1000`, `1_000_000`, `0xFF`, `0b1010`
`float`	ไม่เปลี่ยนแปลงรูป (Immutable)	จุดทศนิยมความละเอียดคู่มาตรฐาน IEEE 754 (64 บิต แม่นยำประมาณ 15-17 ตำแหน่งทศนิยม)	`3.14159`, `-0.001`, `2.5e-4`
`complex`	ไม่เปลี่ยนแปลงรูป (Immutable)	จำนวนเชิงซ้อนประกอบด้วยส่วนจริงและส่วนจินตภาพที่เป็นชนิด `float`	`3 + 4j`, `2.5j`
`bool`	ไม่เปลี่ยนแปลงรูป (Immutable)	คลาสย่อยของ `int` แสดงสถานะความจริงทางตรรกศาสตร์	`True` (มีค่าเทียบเท่า 1), `False` (0)

2.3 กลไกการหาร: การหารจริง (True Division) เทียบกับการหารปัดเศษลง (Floor Division)

Python มีหลักการคำนวณทางคณิตศาสตร์เกี่ยวกับการหารที่แตกต่างจากภาษา C ดังนี้:

- **การหารจริง (True Division: `/`)**: จะคืนค่าเป็นทศนิยม (`float`) เสมอ แม้ว่าตัวตั้งและตัวหารจะเป็นจำนวนเต็มที่หารลงตัวก็ตาม เช่น `6 / 2 → 3.0`

- **การหารปัดเศษลง (Floor Division: `//`)**: ทำการหารและปัดเศษลงเข้าหาค่าอนันต์ทางลบ ($-\infty$):
  - `7 // 2 → 3`
  - `-7 // 2 → -4` (สังเกตว่าไม่เหมือนการตัดทศนิยมทิ้งเข้าหาศูนย์ในภาษา C แต่ Python จะปัดลงสู่ลบอนันต์เสมอ)
- **ตัวดำเนินการมอดุโล (`%`)**: คำนวณเศษที่เหลือตามความสัมพันธ์ทางคณิตศาสตร์: `(a // b) * b + (a % b) == a`
- **การยกกำลัง (`**`)**: ตัวดำเนินการยกกำลังโดยกำเนิด: `2 ** 10 → 1024`

บทที่ 3

คำสั่งแยกสาขาเงื่อนไข & ความหมายเชิงตรรกะของบูลีน

3.1 การทดสอบค่าความจริง: นิยามของ "Truthiness" และ "Falsiness"

ใน Python วัตถุทุกชนิดสามารถนำมาทดสอบค่าความจริงภายในนิพจน์เงื่อนไข `if` หรือ `while` ได้ โดยวัตถุจะถูกประเมินค่าเป็น **เท็จ (Falsy)** ก็ต่อเมื่อเข้าข่ายกรณีใดกรณีหนึ่งดังต่อไปนี้เท่านั้น:

- ค่าคงที่ที่นิยามว่าเป็นเท็จโดยเฉพาะ: `None` และ `False`
- ค่าศูนย์ของชนิดข้อมูลตัวเลขทุกชนิด: `0` , `0.0` , `0j` , `Decimal(0)`
- คอลเลกชันหรือลำดับข้อมูลที่ว่างเปล่า: `""` (สตริงว่าง), `()` (ทูเพิลว่าง), `[]` (ลิสต์ว่าง), `{}` (ดิกชันนารีว่าง), `set()` (เซตว่าง)
- อ็อบเจกต์ที่สร้างขึ้นเองซึ่งมีการนิยามเมธอด `__bool__()` หรือ `__len__()` ที่คืนค่ากลับมาเป็น `False` หรือ `0`

วัตถุอื่น ๆ ทั้งหมดที่นอกเหนือจากกรณีข้างต้น จะถือว่าเป็นจริง (Truthy) โดยปริยาย

3.2 เอกลักษณ์ของวัตถุ (is) เทียบกับความเท่ากันของค่า (==)

ข้อผิดพลาดที่พบบ่อยในหมู่นักพัฒนาคือการสับสนระหว่างการตรวจสอบความเท่ากันของข้อมูลกับการตรวจสอบเอกลักษณ์ของหน่วยความจำ:

- **ความเท่ากันของค่า (Value Equality: `==`)**: ตรวจสอบว่าวัตถุทั้งสองถือครองข้อมูลที่มีค่าเท่ากันหรือไม่ (ทำงานโดยการเรียกเมธอด `__eq__()` ของวัตถุ)
- **เอกลักษณ์ของวัตถุ (Object Identity: `is`)**: ตรวจสอบว่าตัวแปรทั้งสองชี้ไปยัง **แอดเดรสหน่วยความจำตำแหน่งเดียวกันทุกประการหรือไม่** (เปรียบเทียบค่า `id(a) == id(b)`)

```
x = [1, 2, 3]
y = [1, 2, 3]

print(x == y)  # True: ข้อมูลภายในลิสต์มีค่าเท่ากันทุกตำแหน่ง
print(x is y)  # False: วัตถุทั้งสองถูกจัดสรรแยกกันบนหน่วยความจำฮีปคนละตำแหน่ง!

# การเปรียบเทียบกับ None ตามหลักสำนวนไพทอนิค: ต้องใช้ 'is\ หรือ 'is not\ เสมอ
if x is not None:
    print("x is a valid object")
```

3.3 ตัวดำเนินการเชิงตรรกะและการประเมินผลลัพธ์แบบลัดวงจร (Short-Circuiting)

ตัวดำเนินการตรรกะ `and` และ `or` มีการประเมินผลแบบลัดวงจร (Short-Circuit Evaluation) และที่สำคัญยิ่งคือ ใน Python ตัวดำเนินการเหล่านี้ไม่ได้คืนค่าเพียงบูลีน `True` หรือ `False` เท่านั้น **แต่จะคืนค่าเป็นตัววัตถุที่กำหนดผลลัพธ์สุดท้ายจริง ๆ**:

- `x and y` : หาก `x` เป็นเท็จ (Falsy) จะคืนค่า `x` ทันที; หากไม่ใช่ จะประเมินและคืนค่า `y`
- `x or y` : หาก `x` เป็นจริง (Truthy) จะคืนค่า `x` ทันที; หากไม่ใช่ จะประเมินและคืนค่า `y`

สำนวนไพทอนิค: การกำหนดค่าปริยายเริ่มต้น (Default Fallback)

เราสามารถใช้ตัวดำเนินการ `or` ในการกำหนดค่าปริยายได้อย่างกระชับและสง่างาม:

`user_name = input("Name: ") or "Anonymous"`

3.4 การแยกสาขาเงื่อนไข: if-elif-else และการจับคู่รูปแบบเชิงโครงสร้าง

คำสั่งเงื่อนไขพื้นฐานใช้โครงสร้าง `if` , `elif` , และ `else` นอกจากนี้ใน Python 3.10 ได้มีการเพิ่ม **การจับคู่รูปแบบเชิงโครงสร้าง (Structural Pattern Matching: `match - case`)** ซึ่งสามารถจับคู่ได้ทั้งค่าคงที่และโครงสร้างข้อมูลที่ซ้อนกัน:

```
def handle_command(command: dict):
    match command:
        case {"action": "move", "direction": ("north" | "south" | "east" | "west") as d}:
            print(f"Moving agent {d}")
        case {"action": "sleep", "duration": int(sec)} if sec > 0:
            print(f"Sleeping for {sec} seconds")
        case _:
            print("Unknown or invalid command structure")
```

บทที่ 4

การวนซ้ำ, ลำดับข้อมูล & ฟังก์ชัน range()

4.1 สถาปัตยกรรมการวนซ้ำผ่านลำดับข้อมูลด้วยคำสั่ง for-in

คำสั่ง `for` ในภาษา Python มีพื้นฐานเป็น **For-Each Iterator** แตกต่างจากการวนลูปนับตัวเลขแบบดั้งเดิมของภาษา C โดยคำสั่งนี้จะทำการท่องผ่านสมาชิกทุกตัวในคอลเลกชันที่วนซ้ำได้ (Strings, Lists, Tuples, Dictionaries, Sets, Generators) โดยไม่ต้องจัดการดัชนีด้วยตนเอง:

```
message = "PYTHON"
for char in message:
    print(char, end="-")
print()
```

4.2 ฟังก์ชัน range(): การสร้างลำดับช่วงครึ่งเปิด (Half-Open Intervals)

ฟังก์ชันบิลต์อิน `range()` ทำหน้าที่สร้างลำดับตัวเลขที่ไม่เปลี่ยนแปลงรูป โดยรับอาร์กิวเมนต์ได้สูงสุด 3 ตัว:

`range(start, stop, step)` → ช่วง `[start, stop)` โดยเพิ่มทีละก้าว `'step'`

- `range(n)` : สร้างลำดับ $0, 1, 2, \dots, n-1$ (ค่าเริ่มต้น: `start=0` , `step=1`)
- `range(start, stop)` : สร้างค่าตั้งแต่ `start` จนถึงค่าน้อยกว่า `stop` (**ไม่รวมค่า** `stop` เป็นช่วงครึ่งเปิดทางขวา)
- `range(start, stop, step)` : กำหนดระยะก้าวเพิ่มทีละ `step` โดยสามารถนับถอยหลังได้ด้วยค่าก้าวติดลบ เช่น `range(10, 0, -2) → 10, 8, 6, 4, 2`

> **ประสิทธิภาพการใช้หน่วยความจำของ range() ใน Python 3**
> `range()` ไม่ได้สร้างลิสต์ตัวเลขขึ้นมาเก็บไว้ในแรมทั้งหมดพร้อมกัน แต่เป็นอ็อบเจกต์ลำดับขนาดเล็ก (Sequence Object) ที่คำนวณค่าแต่ละตัวเมื่อต้องการใช้งาน ทำให้ใช้พื้นที่หน่วยความจำคงที่ $O(1)$ เสมอ ไม่ว่าช่วงตัวเลขจะครอบคลุม 10 ตัวหรือ 1 หมื่นล้านตัวก็ตาม!

4.3 การควบคุมลูปและโครงสร้าง else ของลูปใน Python

Python มีคีย์เวิร์ดควบคุมการทำงานของลูปตามมาตรฐาน:

- `break` : สั่งยุติการทำงานของลูปชั้นในสุดทันที
- `continue` : ข้ามคำสั่งที่เหลือในรอบปัจจุบัน และเริ่มรอบการวนซ้ำถัดไปทันที
- **ประโยค `else` ของลูป: จะทำงานก็ต่อเมื่อการวนลูปสิ้นสุดลงตามธรรมชาติโดยไม่มีการสั่ง `break` เกิดขึ้น!** นับเป็นคุณลักษณะเฉพาะตัวที่ทรงพลังอย่างยิ่งในการค้นหาข้อมูล:

```
# การหาจำนวนเฉพาะโดยใช้โครงสร้าง loop-else
for n in range(2, 15):
    for d in range(2, int(n ** 0.5) + 1):
        if n % d == 0:
            break # พบตัวหารลงตัว จึงไม่ใช่จำนวนเฉพาะ
    else:
        # ลูปทำงานจนครบโดยไม่เจอ break: รับประกันได้ว่า n เป็นจำนวนเฉพาะแน่นอน!
        print(f"{n} is prime", end=" | ")
print()
```

```
ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)
2 is prime | 3 is prime | 5 is prime | 7 is prime | 11 is prime | 13 is prime |
```

4.4 การเข้าถึงดัชนีและการท่องข้อมูลคู่ขนาน: enumerate() และ zip()

ตามแนวทางไพทอนิค ควรหลีกเลี่ยงการเขียนลูปแบบ `for i in range(len(lst)):` โดยหันมาใช้ฟังก์ชันตัวช่วยขั้นสูงที่ Python จัดเตรียมไว้ให้:

- `enumerate(iterable, start=0)` : คืนค่าเป็นคู่ของ `(index, element)` ในแต่ละรอบ
- `zip(*iterables)` : ทำการท่องผ่านหลายคอลเลกชันพร้อมกันแบบขนานสอดประสานกันทีละคู่

```python
students = ["Alice", "Bob", "Charlie"]
scores = [92, 85, 98]

for rank, (name, score) in enumerate(zip(students, scores), start=1):
    print(f"Rank {rank}: {name} - {score} pts")
```

บทที่ 5

สตริง & กลไกการจัดการข้อความ

5.1 ความไม่เปลี่ยนรูปของสตริง & การสไลซ์แบบขยาย (Extended Slicing)

ในภาษา Python สตริง (`str`) คือลำดับที่จัดเรียงตามลำดับของอักขระยูนิโค้ด (Unicode) คุณลักษณะสำคัญพื้นฐานที่สุดของสตริงใน Python คือ **ความไม่เปลี่ยนรูป (Immutability)**: เมื่อสตริงถูกสร้างขึ้นแล้ว อักขระแต่ละตำแหน่งจะไม่สามารถแก้ไขค่าได้โดยตรง (การสั่ง `s[0] = 'H'` จะทำให้เกิดข้อผิดพลาด `TypeError`)

Python มีไวยากรณ์ **การสไลซ์แบบขยาย (Extended Slicing)** ที่ทรงพลังซึ่งประกอบด้วย 3 ส่วนประกอบหลัก:

`sequence[start : stop : step]`

- ดัชนีติดลบใช้สำหรับนับย้อนหลังจากส่วนท้าย: `s[-1]` คืออักขระตัวสุดท้าย
- การละเว้น `start` จะมีค่าเริ่มต้นเป็น `0` ; การละเว้น `stop` จะมีค่าเริ่มต้นเป็นความยาวของลำดับข้อมูลนั้น
- ค่าก้าวติดลบ (Negative Step) จะทำการท่องข้อมูลย้อนกลับ: `s[::-1]` **สามารถใช้กลับด้านสตริงทั้งหมดได้อย่างมีประสิทธิภาพ!**

```
text = "University of Technology"
print(text[0:10])    # "University"
print(text[-10:])    # "Technology"
print(text[::2])     # "Uiest fTcnlg" (เลือกทีละทุก ๆ 2 ตัวอักษร)
print(text[::-1])    # กลับทิศทางสตริงทั้งหมดจากหลังมาหน้า
```

5.2 เมธอดของสตริงที่ใช้งานบ่อย

การแปลงหรือแก้ไขสตริงจะคืนค่ากลับมาเป็นสตริงใหม่ที่จัดสรรบนหน่วยความจำเสมอ:

เมธอด	คำอธิบายการทำงาน	ตัวอย่างการใช้งาน
`strip([chars])`	ตัดช่องว่างหรืออักขระที่ระบุออกจากส่วนหัวและส่วนท้าย	`" hello ".strip() → "hello"`
`split(sep=None)`	แยกสตริงออกเป็นลิสต์ของคำตามตัวคั่นที่กำหนด	`"a,b,c".split(",") → [a, b, c]`
`join(iterable)`	เชื่อมต่อลำดับของสตริงเข้าด้วยกันโดยใช้สตริงปัจจุบันเป็นตัวเชื่อม	`"-".join([2026, 10, 16]) → "2026-10-16"`
`replace(old, new)`	แทนที่ข้อความย่อยเดิมด้วยข้อความย่อยใหม่	`"cat dog".replace("cat", "cow") → "cow dog"`
`find(sub)`	คืนค่าดัชนีตำแหน่งแรกที่พบข้อความย่อย หรือคืนค่า `-1` หากไม่พบ	`"python".find("th") → 2`
`startswith(prefix)`	คืนค่า `True` หากสตริงเริ่มต้นด้วยคำนำหน้าที่กำหนด	`"script.py".startswith("sc") → True`

บทที่ 6

ลิสต์, ทูเพิล & การสไลซ์ลำดับข้อมูล

6.1 ลิสต์: ลำดับข้อมูลแบบเปลี่ยนแปลงได้และปรับขนาดอัตโนมัติ

ลิสต์ (`list`) คือลำดับข้อมูลที่จัดเรียงตามลำดับ สามารถปรับขนาดได้แบบไดนามิก และสามารถแก้ไขเปลี่ยนแปลงข้อมูลภายในได้ (Mutable) โดยรองรับสมาชิกที่มีชนิดข้อมูลหลากหลาย ภายใน CPython ลิสต์ถูกสร้างขึ้นในรูปแบบของอาร์เรย์พอยน์เตอร์อ็อบเจกต์ที่มีการขยายขนาดอัตโนมัติ

การดำเนินการ	ไวยากรณ์ / เมธอด	ความซับซ้อนเชิงเวลา	คำอธิบายการทำงาน
ต่อท้ายลิสต์ (Append)	`lst.append(x)`	$O(1)$ amortized	เพิ่มสมาชิกใหม่เข้าไปที่ขอบขวาสุดของลิสต์
นำตัวท้ายออก (Pop)	`lst.pop()`	$O(1)$	ลบและคืนค่าสมาชิกตัวขวาสุดออกมา
แทรก ณ ดัชนีที่กำหนด	`lst.insert(i, x)`	$O(n)$	เลื่อนพอยน์เตอร์ของสมาชิกถัดไปทั้งหมดไปทางขวา
ลบ ณ ดัชนีที่กำหนด	`del lst[i]` / `pop(i)`	$O(n)$	เลื่อนพอยน์เตอร์ของสมาชิกถัดไปทั้งหมดกลับมาทางซ้าย
ขยายลิสต์ (Extend)	`lst.extend(iterable)`	$O(k)$	นำสมาชิกทั้งหมดจากอีกคอลเลกชันมาต่อท้าย
จัดเรียงข้อมูลในตัว	`lst.sort(key=..., reverse=...)`	$O(n \log n)$	จัดเรียงข้อมูลด้วยอัลกอริทึม Timsort ประสิทธิภาพสูง

6.2 ทูเพิล: เรกคอร์ดที่ไม่เปลี่ยนแปลงรูปและการ Unpacking

ทูเพิล (`tuple`) คือลำดับข้อมูลที่จัดเรียงตามลำดับแต่ **ไม่สามารถเปลี่ยนแปลงข้อมูลได้ (Immutable)** เมื่อสร้างขึ้นแล้ว จะไม่สามารถเพิ่ม ลบ หรือแก้ไขสมาชิกภายในได้ ด้วยความไม่เปลี่ยนรูปนี้ ทำให้ทูเพิลสามารถนำไปใช้เป็นคีย์ของดิกชันนารีหรือสมาชิกของเซตได้ (ตราบใดที่สมาชิกภายในสามารถแฮชได้)

```
# ทูเพิลที่มีสมาชิกเพียงตัวเดียวต้องมีเครื่องหมายจุลภาค (,) ต่อท้ายเสมอ!
singleton = (42,)   # ชนิดข้อมูล: tuple; หากไม่มีจุลภาค (42) จะเป็นเพียงจำนวนเต็ม int

# การแกะค่าทูเพิล (Tuple Unpacking) ร่วมกับ Star Expressions
point = (10, 20, 30, 40, 50)
x, y, *rest = point  # x=10, y=20, rest=[30, 40, 50]

# การสลับค่าตัวแปรแบบไพทอนิค (ไม่ต้องใช้ตัวแปรชั่วคราว temp!)
a, b = 5, 10
a, b = b, a          # ประเมินทูเพิลฝั่งขวา (10, 5) แล้วกระจายค่ากลับเข้าสู่ a และ b
```

6.3 การทำสำเนาแบบตื้น (Shallow Copy) เทียบกับการทำสำเนาแบบลึก (Deep Copy)

คำสั่งกำหนดค่า (`=`) ใน Python ไม่ได้สร้างสำเนาข้อมูล แต่เป็นการสร้างการอ้างอิงใหม่ไปยังวัตถุเดิม หากต้องการทำสำเนาข้อมูลจริง ๆ ต้องเข้าใจความแตกต่างดังนี้:

- **การทำสำเนาแบบตื้น (Shallow Copy: `copy.copy(obj)` หรือ `lst.copy()`)**: สร้างโครงสร้างคอลเลกชันใหม่ แต่สมาชิกภายในยังคงชี้ไปยังวัตถุย่อยตัวเดิม หากสมาชิกย่อยนั้นเป็นวัตถุที่แก้ไขได้ การเปลี่ยนแปลงจะส่งผลถึงกันทั้งสองฝั่ง!
- **การทำสำเนาแบบลึก (Deep Copy: `copy.deepcopy(obj)`)**: ทำการสำเนาวัตถุซ้ำลงไปในทุกระดับชั้นอย่างสมบูรณ์แบบ ทำให้หน่วยความจำแยกขาดจากกันโดยสิ้นเชิง

บทที่ 7

ดิกชันนารี, เซต & คอลเลกชันแบบแฮช

7.1 ดิกชันนารี: แฮชแมปประสิทธิภาพ O(1)

ดิกชันนารี (`dict`) เป็นโครงสร้างข้อมูลแบบจับคู่ความสัมพันธ์ระหว่างคีย์ที่ไม่ซ้ำกันกับค่าข้อมูล (Key-Value Mapping) ตั้งแต่ Python 3.7 เป็นต้นมา ดิกชันนารีรับประกันการรักษาลำดับตามการแทรกข้อมูล (Insertion Order) พร้อมทั้งมอบความเร็วเฉลี่ยระดับ $O(1)$ สำหรับการค้นหา เพิ่ม และลบข้อมูล:

```
user = {
    "id": 1001,
    "name": "Samantha",
    "roles": ["admin", "engineer"]
}

# การเข้าถึงข้อมูลอย่างปลอดภัยด้วย .get() เพื่อป้องกันข้อผิดพลาด KeyError
role = user.get("department", "Unassigned")

# การวนซ้ำคู่คีย์และค่าข้อมูลผ่าน .items()
for key, value in user.items():
    print(f"{key} -> {value}")
```

7.2 เซต: คอลเลกชันสมาชิกไม่ซ้ำและพีชคณิตเซต

เซต (`set`) เก็บรักษาข้อมูลที่มีเอกลักษณ์เฉพาะตัว ไม่ซ้ำกัน ไม่มีการเรียงลำดับ และต้องเป็นข้อมูลที่แฮชได้ โดยรองรับการตรวจสอบการเป็นสมาชิกในเวลาคงที่ $O(1)$ (`x in my_set`) รวมถึงการดำเนินการทางพีชคณิตเซต:

```
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)   # ยูเนียน (Union): {1, 2, 3, 4, 5, 6}
print(a & b)   # อินเตอร์เซกชัน (Intersection): {3, 4}
print(a - b)   # ผลต่าง (Difference): {1, 2} (สมาชิกที่อยู่ใน a แต่ไม่อยู่ใน b)
print(a ^ b)   # ผลต่างสมมาตร (Symmetric Difference): {1, 2, 5, 6}
```

บทที่ 8

คอมพรีเฮนชัน & นิพจน์เจเนอเรเตอร์

8.1 คอมพรีเฮนชันสำหรับลิสต์, ดิกชันนารี และเซต

คอมพรีเฮนชัน (Comprehensions) ช่วยให้สามารถสร้างและกรองข้อมูลในคอลเลกชันได้อย่างกระชับ อ่านง่าย และมีความเร็วสูงโดยไม่ต้องเขียนลูปขั้นตอนแบบดั้งเดิม:

```
# 1. List Comprehension: [expression for item in iterable if condition]
numbers = [1, 2, 3, 4, 5, 6]
even_squares = [x ** 2 for x in numbers if x % 2 == 0]
# ผลลัพธ์: [4, 16, 36]

# 2. Dictionary Comprehension: {key_expr: val_expr for ...}
words = ["apple", "banana", "cherry"]
word_lengths = {w: len(w) for w in words}
# ผลลัพธ์: {apple: 5, banana: 6, cherry: 6}

# 3. Set Comprehension: {expr for ...}
unique_remainders = {x % 3 for x in numbers}
# ผลลัพธ์: {0, 1, 2}
```

8.2 นิพจน์เจเนอเรเตอร์: การประเมินผลแบบหน่วงเวลา (Lazy Evaluation)

การเปลี่ยนวงเล็บเหลี่ยมของ List Comprehension ให้เป็นวงเล็บกลมจะทำให้ได้ **นิพจน์เจเนอเรเตอร์ (Generator Expression)** แทนที่จะจัดสรรหน่วยความจำสำหรับเก็บข้อมูลทั้งหมดในคราวเดียว เจเนอเรเตอร์จะประเมินค่าข้อมูลทีละตัวตามความต้องการ (On Demand):

```
# สิ้นเปลืองหน่วยความจำแรมหลายเมกะไบต์เพื่อสร้างสมาชิกหลายล้านตัวพร้อมกัน:
huge_list = [x ** 2 for x in range(10_000_000)]

# ใช้หน่วยความจำแรมเพียงไม่กี่ไบต์ด้วยนิพจน์เจเนอเรเตอร์:
huge_gen = (x ** 2 for x in range(10_000_000))

# สตรีมข้อมูลเข้าสู่ sum() โดยตรงโดยไม่ต้องพักตัวเลขทั้งหมดไว้ในแรม!
total = sum(x ** 2 for x in range(1000))
print(f"Sum of squares: {total}")
```

บทที่ 9

กลไกของฟังก์ชัน, พารามิเตอร์ & สโคป (LEGB)

9.1 ชนิดของพารามิเตอร์: ตำแหน่ง, คีย์เวิร์ด, *args และ **kwargs

ฟังก์ชันในภาษา Python ถูกนิยามด้วยคีย์เวิร์ด `def` และรองรับกลไกการส่งผ่านอาร์กิวเมนต์ที่ยืดหยุ่นและหลากหลาย:

- **พารามิเตอร์ตามตำแหน่ง (Positional Arguments)**: ถูกจับคู่กับพารามิเตอร์ตามลำดับจากซ้ายไปขวา
- **พารามิเตอร์ปริยาย / ตามชื่อคีย์เวิร์ด (Default / Keyword Arguments)**: พารามิเตอร์ที่มีการกำหนดค่าเริ่มต้นสำรองไว้
- **พารามิเตอร์ตำแหน่งแบบไม่จำกัดจำนวน (`*args`)**: รวบรวมอาร์กิวเมนต์ตามตำแหน่งส่วนเกินทั้งหมดให้อยู่ในรูปของ `tuple`
- **พารามิเตอร์คีย์เวิร์ดแบบไม่จำกัดจำนวน (`**kwargs`)**: รวบรวมอาร์กิวเมนต์แบบระบุชื่อส่วนเกินทั้งหมดให้อยู่ในรูปของ `dict`
- **พารามิเตอร์ที่บังคับระบุชื่อเท่านั้น (Keyword-Only Arguments)**: พารามิเตอร์ที่อยู่ถัดจากเครื่องหมาย `*` เดี่ยว ๆ จะต้องถูกส่งค่าด้วยการระบุชื่อคีย์เวิร์ดอย่างชัดเจนเท่านั้น

```python
def configure_server(host: str, port: int = 8080, *flags, timeout: int = 30, **options):
    print(f"Server: {host}:{port} | Timeout: {timeout}s")
    print(f"Flags: {flags} | Options: {options}")

configure_server("localhost", 9000, "--debug", "--verbose", timeout=60, ssl=True, cert="/etc/ssl")
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Server: localhost:9000 | Timeout: 60s
Flags: ('--debug', '--verbose') | Options: {'ssl': True, 'cert': '/etc/ssl'}
```

9.2 กฎการค้นหาขอบเขตสโคป LEGB

เมื่อมีการอ้างถึงชื่อตัวแปรภายในฟังก์ชัน ตัวแปลภาษา Python จะทำการค้นหาขอบเขตสโคปตามลำดับชั้น 4 ระดับอย่างเคร่งครัด (**กฎ LEGB**):

1. **L (Local)**: ชื่อตัวแปรที่ถูกกำหนดค่าภายในขอบเขตฟังก์ชันปัจจุบัน
2. **E (Enclosing)**: ชื่อตัวแปรในฟังก์ชันภายนอกที่ครอบฟังก์ชันปัจจุบันอยู่ (จากชั้นในสุดออกสู่ชั้นนอกสุด)
3. **G (Global)**: ชื่อตัวแปรที่ถูกประกาศไว้ที่ระดับบนสุดของมอดูลไฟล์ปัจจุบัน
4. **B (Built-in)**: ชื่อมาตรฐานที่โหลดไว้ล่วงหน้าในเนมสเปซของ Python (เช่น `len` , `range` , `print`)

คำสั่งปรับเปลี่ยนสโคป: global และ nonlocal

- `global x` : แจ้งให้คอมไพเลอร์ทราบว่าการกำหนดค่าตัวแปร `x` เป็นการเปลี่ยนแปลงตัวแปรระดับโกลบอลของมอดูล
- `nonlocal x` : แจ้งให้คอมไพเลอร์ทราบว่าการกำหนดค่า `x` เป็นการแก้ไขตัวแปรของฟังก์ชันภายนอกที่ครอบอยู่ลำดับถัดไป (สำคัญอย่างยิ่งสำหรับ Closures)

กับดักคลาสสิกของ Python: อาร์กิวเมนต์ปริยายที่แก้ไขได้ (Mutable Default Arguments)

นิพจน์ค่าปริยายของพารามิเตอร์จะถูกประเมินค่า **เพียงครั้งเดียวในขณะที่มีการนิยามฟังก์ชันเท่านั้น** ไม่ใช่ทุกครั้งที่มีการเรียกใช้ฟังก์ชัน! ดังนั้นหากใช้วัตถุที่เปลี่ยนแปลงค่าได้ (เช่น list หรือ dict) เป็นค่าปริยาย วัตถุชิ้นนั้นจะถูกแชร์ร่วมกันตลอดการเรียกใช้งานทุกครั้ง!

```
# พฤติกรรมที่ไม่พึงประสงค์: lst จะสะสมข้อมูลเพิ่มขึ้นเรื่อย ๆ ข้ามการเรียกฟังก์ชันแต่ละครั้ง!
def bad_append(val, lst=[]):
    lst.append(val)
    return lst

# แนวทางแก้ไขที่ถูกต้องตามแบบแผนไพทอนิค: ใช้ None เป็นตัวแทนตรวจสอบ (Sentinel)
def safe_append(val, lst=None):
    if lst is None:
        lst = []
    lst.append(val)
    return lst
```

บทที่ 10

การโปรแกรมเชิงฟังก์ชัน: แรมบ์ดา, ฟังก์ชันอันดับสูง & อิตเทอเรเตอร์

10.1 First-Class Functions และฟังก์ชันนิรนาม (Lambda)

ในภาษา Python ฟังก์ชันได้รับการยกสถานะเป็น **First-Class Objects**: สามารถกำหนดให้กับตัวแปร ส่งเป็นอาร์กิวเมนต์ให้กับฟังก์ชันอื่น จัดเก็บไว้ในโครงสร้างข้อมูล และคืนค่ากลับออกมาจากฟังก์ชันได้เฉกเช่นเดียวกับวัตถุทั่วไป

คีย์เวิร์ด `lambda` ใช้สำหรับสร้างฟังก์ชันนิรนามขนาดกะทัดรัดแบบอินไลน์ที่จำกัดเพียงนิพจน์เดียว:

`lambda param1, param2: expression`

```python
# การจัดเรียงข้อมูลทูเพิลเชิงซ้อนตามคีย์ที่กำหนดด้วยฟังก์ชัน lambda
employees = [
    ("Alice", "Engineering", 95000),
    ("Bob", "Marketing", 72000),
    ("Charlie", "Engineering", 110000)
]

# จัดเรียงจากเงินเดือนมากไปหาน้อย (ดัชนี 2)
sorted_by_salary = sorted(employees, key=lambda emp: emp[2], reverse=True)
for emp in sorted_by_salary:
    print(f"{emp[0]} ({emp[1]}): ${emp[2]:,}")
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
Charlie (Engineering): $110,000
Alice (Engineering): $95,000
Bob (Marketing): $72,000
```

10.2 ฟังก์ชันเจเนอเรเตอร์และคำสั่ง yield

ฟังก์ชันเจเนอเรเตอร์ (Generator Function) จะใช้คำสั่ง `yield` แทนการ `return` เมื่อโปรแกรมพบคำสั่ง `yield` ฟังก์ชันจะหยุดพักสถานะการทำงานทั้งหมดไว้ชั่วคราวและส่งค่ากลับไปยังผู้เรียก เมื่อตัววนซ้ำ (Iterator) มีการขอรับค่าตัวถัดไป ฟังก์ชันจะกลับมาทำงานต่อจากจุดที่สั่ง `yield` ไว้ทันที:

```python
def fibonacci_stream(limit: int):
    """ฟังก์ชันเจเนอเรเตอร์สำหรับสร้างลำดับฟีโบนักชีแบบหน่วงเวลาประหยัดหน่วยความจำ"""
    a, b = 0, 1
    for _ in range(limit):
        yield a
        a, b = b, a + b

# การสร้างสตรีมข้อมูลที่ใช้หน่วยความจำระดับคงที่ O(1)
for val in fibonacci_stream(8):
    print(val, end=" ")
print()
```

บทที่ 11

เดคอเรเตอร์, โคลเชอร์ & แรปเปอร์ฟังก์ชัน

11.1 กลไกของโคลเชอร์และเดคอเรเตอร์

โคลเชอร์ (Closure) เกิดขึ้นเมื่อฟังก์ชันชั้นในสามารถเข้าถึงและจดจำตัวแปรที่อยู่ในขอบเขตของฟังก์ชันชั้นนอกได้ แม้ว่าฟังก์ชันชั้นนอกจะประมวลผลเสร็จสิ้นไปแล้วก็ตาม

เดคอเรเตอร์ (Decorator) คืออ็อบเจกต์ที่เรียกใช้ได้ (Callable) ซึ่งรับฟังก์ชันเข้ามาเป็นอาร์กิวเมนต์ แล้วคืนค่าเป็นฟังก์ชันที่ได้รับการห่อหุ้มหรือเพิ่มขยายความสามารถกลับออกไป โดยไวยากรณ์ `@decorator` เป็นเพียง Syntactic Sugar ที่มีความหมายดังนี้:

`@my_decorator ≡ func = my_decorator(func)`

```python
import time
from functools import wraps

def performance_timer(func):
    """เดคอเรเตอร์สำหรับจับเวลาการประมวลผลของฟังก์ชัน"""
    @wraps(func) # สำคัญยิ่ง: ช่วยรักษาชื่อเดิมและ docstring ของฟังก์ชันเป้าหมาย
    def wrapper(*args, **kwargs):
        start_time = time.perf_counter()
        result = func(*args, **kwargs)
        duration = time.perf_counter() - start_time
        print(f"[TIMER] {func.__name__}() took {duration:.6f} seconds")
        return result
    return wrapper

@performance_timer
def heavy_computation(n: int) -> int:
    return sum(i ** 2 for i in range(n))

val = heavy_computation(500_000)
```

บทที่ 12

มอดูล, แพ็กเกจ & สถาปัตยกรรมโครงงาน

12.1 ระบบมอดูลและการป้องกันจุดเริ่มต้นโปรแกรม (Entry Point Guard)

ใน Python **มอดูล (Module)** คือไฟล์ `.py` ที่บรรจุโค้ดคำสั่ง ส่วน **แพ็กเกจ (Package)** คือไดเรกทอรีที่ประกอบด้วยมอดูลหลายตัวและอาจมีไฟล์ `__init__.py` กำกับอยู่

เมื่อ Python ประมวลผลสคริปต์ ระบบจะกำหนดค่าให้กับตัวแปรพิเศษ `__name__` :

- หากไฟล์ถูกสั่งรันโดยตรงผ่านบรรทัดคำสั่ง: `__name__ == "__main__"`
- หากไฟล์ถูกนำเข้า (Import) โดยมอดูลอื่น: `__name__` จะมีค่าเป็นชื่อเส้นทางการนำเข้าของมอดูลนั้น

แบบแผนมาตรฐานของจุดเริ่มต้นโปรแกรมหลัก (Main Entry Point)

ควรกำกับบล็อกคำสั่งที่ต้องการให้ทำงานเฉพาะตอนสั่งรันโดยตรงด้วยคำสั่งมาตรฐานเสมอ:

```
if __name__ == "__main__":
    main()
```

วิธีนี้ช่วยให้ฟังก์ชันภายในไฟล์สามารถถูกนำเข้า (Import) ไปใช้งานหรือทำ Unit Test ในไฟล์อื่นได้อย่างปลอดภัยโดยไม่ก่อให้เกิดผลข้างเคียงที่ไม่พึงประสงค์

บทที่ 13

หลักการเชิงวัตถุ: คลาส, แอตทริบิวต์ & เมธอด

13.1 กลไกของคลาส: เมธอดสร้างอินสแตนซ์และพารามิเตอร์ self

ภาษา Python ขับเคลื่อนด้วยกระบวนทัศน์เชิงวัตถุอย่างสมบูรณ์แบบ: ทุกสิ่งตั้งแต่จำนวนเต็มไปจนถึงฟังก์ชันล้วนเป็นอินสแตนซ์ของคลาส การประกาศคลาสทำได้ผ่านคีย์เวิร์ด `class` เมื่อมีการสร้างอินสแตนซ์ขึ้นมา Python จะเรียกเมธอด `__init__()` โดยส่งอินสแตนซ์ที่เพิ่งสร้างเสร็จใหม่ ๆ เข้ามาเป็นอาร์กิวเมนต์ตัวแรก ซึ่งตามธรรมเนียมนิยมจะตั้งชื่อว่า `self`:

```
class BankAccount:
    # Class Attribute: ตัวแปรระดับคลาสที่แชร์ร่วมกันในทุกอินสแตนซ์
    bank_name: str = "Global Trust Bank"
    interest_rate: float = 0.035

    def __init__(self, account_holder: str, initial_balance: float = 0.0):
        # Instance Attributes: คุณลักษณะเฉพาะตัวของแต่ละอินสแตนซ์
        self.holder = account_holder
        self._balance = initial_balance  # ขีดล่างเดี่ยว: ข้อตกลงว่าเป็นข้อมูลสงวน (Protected)

    def deposit(self, amount: float) -> None:
        if amount <= 0:
            raise ValueError("Deposit amount must be positive")
        self._balance += amount

    # การจัดการแอตทริบิวต์ผ่าน @property (Getter)
    @property
    def balance(self) -> float:
        return self._balance
```

13.2 การจำแนกประเภทเมธอด: Instance, Class และ Static Methods

Python จัดแบ่งประเภทของเมธอดออกเป็น 3 ระดับโครงสร้างอย่างชัดเจน:

ประเภทเมธอด	เดคอเรเตอร์	พารามิเตอร์แรก	สิทธิ์การเข้าถึงและจุดประสงค์การใช้งาน
Instance Method	ไม่มี (ค่าปริยาย)	`self`	เข้าถึงและแก้ไขสถานะของอินสแตนซ์และสถานะของคลาสได้โดยตรง
Class Method	`@classmethod`	`cls`	เข้าถึงและแก้ไขสถานะของคลาส; นิยมใช้สร้าง Constructor ทางเลือก (Factory Methods)
Static Method	`@staticmethod`	ไม่มี	ฟังก์ชันอรรถประโยชน์อิสระ ไม่สามารถเข้าถึงหรือแก้ไขสถานะของอินสแตนซ์หรือคลาสได้

```python
class Temperature:
    def __init__(self, celsius: float):
        self.celsius = celsius

    # ตัวสร้างอินสแตนซ์ทางเลือกแบบ Factory Constructor
    @classmethod
    def from_fahrenheit(cls, fahrenheit: float):
        celsius = (fahrenheit - 32) * 5 / 9
        return cls(celsius)

    # เมธอดอรรถประโยชน์แบบสถิต
    @staticmethod
    def is_freezing(celsius: float) -> bool:
        return celsius <= 0.0
```

บทที่ 14

การสืบทอด, ภาวะหลายรูป & Dunder Methods พิเศษ

14.1 การสืบทอดคุณสมบัติ, ฟังก์ชัน super() และลำดับการค้นหา MRO

คลาสย่อยจะได้รับการถ่ายทอดแอตทริบิวต์และเมธอดมาจากคลาสแม่ โดยฟังก์ชัน `super()` จะทำหน้าที่ส่งผ่านการเรียกใช้งานไปยังคลาสแม่หรือคลาสลำดับถัดไปในลำดับชั้นอย่างเป็นระเบียบ:

```python
class Entity:
    def __init__(self, name: str):
        self.name = name

class Player(Entity):
    def __init__(self, name: str, health: int = 100):
        super().__init__(name)  # ส่งต่อการกำหนดค่าเริ่มต้นไปยังคลาสแม่
        self.health = health
```

สำหรับกรณีการสืบทอดหลายสาย (Multiple Inheritance) Python จะตัดสินลำดับการค้นหาแอตทริบิวต์ด้วย **อัลกอริทึม C3 Linearization (Method Resolution Order: MRO)** ซึ่งสามารถตรวจสอบลำดับได้ผ่าน `ClassName.mro()` หรือ `ClassName.__mro__`

14.2 โมเดลข้อมูลของ Python: เมธอดมหัศจรรย์ (Dunder Methods)

ในภาษา Python เราไม่จำเป็นต้องเขียนเมธอดแบบภาษา Java เช่น `add()` หรือ `toString()` แต่ภาษา Python ได้จัดเตรียม **เมธอดพิเศษ (Special / Dunder Methods)** ซึ่งขึ้นต้นและลงท้ายด้วยขีดล่างคู่ไว้ให้ การเขียนโค้ดรองรับเมธอด Dunder เหล่านี้จะช่วยให้คลาสที่สร้างขึ้นใหม่สามารถผสานการทำงานร่วมกับไวยากรณ์มาตรฐานของภาษา Python ได้อย่างกลมกลืน (ทั้งตัวดำเนินการ, `len()`, การวนลูป, การสไลซ์ และการแปลงเป็นสตริง):

เมธอด Dunder	ไวยากรณ์ที่กระตุ้นการทำงาน	จุดประสงค์การทำงาน
`__repr__(self)`	`repr(obj)`, หน้าจอเชลล์โต้ตอบ	แสดงข้อความระบุรายละเอียดวัตถุสำหรับนักพัฒนาอย่างไม่กำกวม (เช่น `Vector(x=1, y=2)`)
`__str__(self)`	`str(obj)`, `print(obj)`	แสดงข้อความที่มนุษย์ทั่วไปสามารถอ่านเข้าใจได้ง่าย
`__len__(self)`	`len(obj)`	คืนค่าจำนวนสมาชิกในคอลเลกชัน (ต้องเป็นจำนวนเต็มที่ไม่ติดลบ)
`__getitem__(self, key)`	`obj[key]`	รองรับการเข้าถึงข้อมูลด้วยดัชนีและการสไลซ์
`__add__(self, other)`	`obj1 + obj2`	รองรับตัวดำเนินการบวกทางคณิตศาสตร์
`__eq__(self, other)`	`obj1 == obj2`	รองรับการเปรียบเทียบความเท่ากันของข้อมูล
`__lt__(self, other)`	`obj1 < obj2`	รองรับการเปรียบเทียบน้อยกว่า (ทำให้ใช้งานฟังก์ชัน `sorted()` ได้)

14.3 ตัวอย่างการสร้างคลาสเวกเตอร์ฉบับสมบูรณ์

ตัวอย่างการสร้างเวกเตอร์ทางคณิตศาสตร์ 2 มิติต่อไปนี้แสดงให้เห็นถึงการโอเวอร์โหลดตัวดำเนินการตามแบบแผนไพทอนิคอย่างครบถ้วน:

```python
import math

class Vector2D:
    def __init__(self, x: float, y: float):
        self.x = x
        self.y = y

    def __repr__(self) -> str:
        return f"Vector2D({self.x}, {self.y})"

    def __add__(self, other: "Vector2D") -> "Vector2D":
        if not isinstance(other, Vector2D):
            return NotImplemented
        return Vector2D(self.x + other.x, self.y + other.y)

    def __mul__(self, scalar: float) -> "Vector2D":
        return Vector2D(self.x * scalar, self.y * scalar)

    def magnitude(self) -> float:
        return math.hypot(self.x, self.y)

v1 = Vector2D(3, 4)
v2 = Vector2D(1, 2)
v3 = v1 + v2
print(f"v1 + v2 = {v3} | Magnitude: {v1.magnitude():.1f}")
```

ผลลัพธ์การรันคำสั่ง (EXECUTION OUTPUT)

```
v1 + v2 = Vector2D(4, 6) | Magnitude: 5.0
```

บทที่ 15

การจัดการข้อยกเว้น & การออกแบบเพื่อความทนทาน

15.1 โครงสร้างการจัดการข้อยกเว้นฉบับสมบูรณ์

ซอฟต์แวร์ระดับมืออาชีพต้องสามารถคาดการณ์และฟื้นฟูระบบจากข้อผิดพลาดขณะรันไทม์ได้ผ่านกลไกการดักจับข้อยกเว้นของ Python โดยโครงสร้างแบบสมบูรณ์จะประกอบด้วย 4 ประโยคคำสั่งที่ทำงานประสานกัน:

- `try` : ห่อหุ้มคำสั่งที่มีโอกาสเกิดข้อผิดพลาดหรือข้อยกเว้น
- `except ExceptionType as err` : ดักจับและจัดการกับชนิดของข้อยกเว้นที่ระบุอย่างเจาะจง
- `else` : **จะทำงานก็ต่อเมื่อบล็อก `try` ผ่านการประมวลผลฉลุยโดยไม่มีข้อผิดพลาดเกิดขึ้นเลย** (เหมาะสำหรับโค้ดที่ต้องพึ่งพาผลลัพธ์ของ `try`)
- `finally` : **รับประกันว่าจะทำงานอย่างไม่มีเงื่อนไขเสมอ** ไม่ว่าจะมีข้อยกเว้นเกิดขึ้น ดักจับได้ หรือไม่สามารถดักจับได้ก็ตาม (จำเป็นอย่างยิ่งสำหรับการคืนทรัพยากรระบบ ปิดไฟล์ หรือปลดล็อกฐานข้อมูล)

```python
def safe_divide_and_log(a: float, b: float) -> float:
    try:
        result = a / b
    except ZeroDivisionError as err:
        print(f"[ERROR] Cannot divide by zero: {err}")
        return 0.0
    except TypeError as err:
        print(f"[ERROR] Non-numeric operand provided: {err}")
        return 0.0
    else:
        print("[SUCCESS] Calculation completed cleanly.")
        return result
    finally:
        print("[AUDIT] Operation cycle concluded.")
```

15.2 ลำดับชั้นข้อยกเว้นที่กำหนดขึ้นเอง (Custom Exception Hierarchies)

ไลบรารีระดับโปรดักชันจะนิยามข้อยกเว้นเฉพาะของระบบตนเองขึ้นมาโดยการสืบทอดคุณสมบัติมาจากคลาสมาตรฐาน `Exception` :

```python
class DatabaseError(Exception):
    """ข้อยกเว้นระดับรากฐานสำหรับข้อผิดพลาดเกี่ยวกับฐานข้อมูล"""
    pass

class RecordNotFoundError(DatabaseError):
    """ถูกเรียกใช้เมื่อการสืบค้นข้อมูลในฐานข้อมูลไม่พบระเบียนข้อมูลที่ต้องการ"""
    def __init__(self, table: str, record_id: int):
        super().__init__(f"Record {record_id} does not exist in table '{table}'")
        self.table = table
        self.record_id = record_id
```

บทที่ 16

การจัดการไฟล์, Context Managers & การแปลงรูปแบบข้อมูล

16.1 ตัวจัดการบริบท (Context Managers): คำสั่ง with

พึงหลีกเลี่ยงการสั่ง `file.close()` ด้วยตนเองโดยเด็ดขาด! เนื่องจากหากเกิดข้อผิดพลาดขึ้นระหว่างคำสั่ง `open()` และ `close()` จะทำให้ File Descriptor รั่วไหลค้างอยู่ในระบบปฏิบัติการ การใช้คำสั่ง `with` ร่วมกับ **โพรโทคอล Context Manager** (`__enter__` และ `__exit__`) จะรับประกันว่าไฟล์จะถูกปิดอย่างปลอดภัยเสมอเมื่อสิ้นสุดบล็อก แม้จะมีข้อยกเว้นเกิดขึ้นก็ตาม:

```
# การเขียนไฟล์ด้วยรหัสอักขระ UTF-8 อย่างปลอดภัย
with open("server_config.txt", "w", encoding="utf-8") as fp:
    fp.write("HOST=127.0.0.1
PORT=8080
ENVIRONMENT=production
")

# การอ่านไฟล์ทีละบรรทัดอย่างมีประสิทธิภาพและประหยัดแรม
with open("server_config.txt", "r", encoding="utf-8") as fp:
    for line in fp:
        print(line.strip())
```

16.2 การแปลงข้อมูลโครงสร้าง: JSON และ CSV

Python มีมอดูลมาตรฐานสำหรับการแลกเปลี่ยนข้อมูลระดับองค์กรอย่างครบครัน:

```
import json
import csv
from pathlib import Path

# 1. การแปลงข้อมูล JSON (จากดิกชันนารีเป็นไฟล์ JSON ที่จัดรูปแบบสวยงาม)
data = {
    "version": "2.1",
    "metrics": {"uptime": 99.98, "active_users": 1450}
}
with open("metrics.json", "w", encoding="utf-8") as jf:
    json.dump(data, jf, indent=2)

# 2. การจัดการระบบไฟล์ยุคใหม่ด้วยโมดูล Pathlib
workspace_dir = Path(".")
for file_path in workspace_dir.glob("*.json"):
    print(f"Found JSON artifact: {file_path.name} ({file_path.stat().st_size} bytes)")
```

บทที่ 17

สำนวนไพทอนิค, การเพิ่มประสิทธิภาพ & การป้องกันข้อผิดพลาด

17.1 ปรัชญาแห่งไพทอน: The Zen of Python (PEP 20)

ปรัชญาชี้นำเบื้องหลังการออกแบบสถาปัตยกรรมภาษา Python ได้รับการสรุปไว้ใน *The Zen of Python* (สามารถเรียกอ่านได้ผ่านคำสั่ง `import this`):

- ความงดงามย่อมดีกว่าความอัปลักษณ์ *(Beautiful is better than ugly.)*
- ความชัดเจนตรงไปตรงมาย่อมดีกว่าการซ่อนเร้นกำกวม *(Explicit is better than implicit.)*
- ความเรียบง่ายย่อมดีกว่าความซับซ้อน *(Simple is better than complex.)*
- โครงสร้างแบบระนาบเดียวดีกว่าการซ้อนชั้นซับซ้อน *(Flat is better than nested.)*
- ความโปร่งบางสบายตาย่อมดีกว่าความหนาแน่นแออัด *(Sparse is better than dense.)*
- ความสามารถในการอ่านและเข้าใจโค้ดคือหัวใจสำคัญ *(Readability counts.)*
- กรณีพิเศษไม่ควรพิเศษถึงขั้นฝ่าฝืนกฎเกณฑ์หลัก *(Special cases aren't special enough to break the rules.)*
- ข้อผิดพลาดไม่ควรปล่อยให้ผ่านไปเงียบ ๆ เว้นแต่จะระบุชัดเจน *(Errors should never pass silently... Unless explicitly silenced.)*
- ควรมีหนทางเดียวที่เห็นชัดเจนที่สุดในการแก้ปัญหา *(There should be one-- and preferably only one --obvious way to do it.)*

17.2 แกลเลอรีรวบรวมกับดักข้อผิดพลาดที่พบบ่อยใน Python

กับดักข้อผิดพลาด	โค้ดที่มีความเสี่ยง	สาเหตุและผลกระทบ	แนวทางแก้ไขแบบไพทอนิคที่ปลอดภัย
การแก้ไขลิสต์ขณะกำลังวนลูป	`for x in lst: if cond: lst.remove(x)`	ดัชนีภายในลูปจะเลื่อนข้ามสมาชิกตัวถัดไป ทำให้ประมวลผลตกหล่นโดยเงียบงัน!	วนลูปบนสำเนา: `for x in lst[:]:` หรือใช้ List Comprehension แทน
การผูกตัวแปรล่าช้าในโคลเชอร์ (Late-Binding)	`funcs = [lambda: i for i in range(4)]`	แลมบ์ดาจะค้นหาตัวแปร `i` ตามการอ้างอิงตอนเรียกใช้ ทำให้ทุกตัวคืนค่า `3` เท่ากันหมด!	ผูกค่าเป็นพารามิเตอร์ปริยาย: `lambda i=i: i`
การสร้างทูเพิลโดยไม่ตั้งใจ	`val = 10,`	เครื่องหมายจุลภาคต่อท้ายจะแปลง `val` ให้กลายเป็นทูเพิล `(10,)` โดยไม่รู้ตัว	ตัดเครื่องหมายจุลภาคส่วนเกินออก
การตรวจสอบความว่างเปล่าที่เยิ่นเย้อ	`if x is not None and len(x) > 0`	เป็นการเขียนเงื่อนไขที่ซ้ำซ้อนและยาวเกินความจำเป็น	ใช้ประโยชน์จาก Truthiness ตามธรรมชาติ: `if x:`

17.3 สถาปัตยกรรมหน่วยความจำภายใน: การนับการอ้างอิงและ GIL

CPython จัดการหน่วยความจำอัตโนมัติผ่าน 2 กลไกหลักที่ประสานกันอย่างลงตัว:

1. **การนับจำนวนการอ้างอิง (Reference Counting)**: วัตถุทุกตัวใน Python จะเก็บตัวนับการอ้างอิงภายใน (`ob_refcnt`) เมื่อมีการผูกโยงตัวแปรเพิ่ม ตัวนับจะเพิ่มขึ้น; เมื่อตัวแปรหลุดจากขอบเขตหรือถูกลบ (`del`) ตัวนับจะลดลง เมื่อตัวนับลดลงเหลือ `0` หน่วยความจำส่วนนั้นจะถูกคืนให้แก่ระบบทันที
2. **ตัวเก็บขยะแบบวัฏจักร (Cyclic Garbage Collector)**: กลไกนับการอ้างอิงไม่สามารถจัดการกับกรณีการอ้างอิงเป็นวงกลมได้ (เช่น วัตถุ A ชี้ไปยัง B และ B ชี้กลับมาที่ A) ตัว Cyclic GC จึงทำหน้าที่สแกนหาและตัดวงจรขยะที่ตัดขาดจากโปรแกรมหลักเป็นระยะ

3. **Global Interpreter Lock (GIL)**: กลไกมิวเท็กซ์ล็อกระดับเธรดที่ป้องกันไม่ให้เธรดระดับระบบปฏิบัติการหลายตัวประมวลผลไบต์โค้ดของ CPython พร้อมกันในคราวเดียว สำหรับงานประมวลผลหนักทาง CPU จึงควรใช้มอดูล `multiprocessing` หรือ Python 3.13 รุ่น Free-Threaded แทนการใช้ Threading

ภาคผนวก A

ตารางลำดับความสำคัญของตัวดำเนินการในภาษา Python

ลำดับความสำคัญของตัวดำเนินการ (Operator Precedence) เป็นตัวกำหนดลำดับการคำนวณในนิพจน์ที่ไม่มีการใส่วงเล็บ โดยตัวดำเนินการที่อยู่ระดับบนสุดจะมีอำนาจผูกมัดสูงสุด และตัวดำเนินการที่อยู่ระดับล่างสุดจะมีอำนาจผูกมัดต่ำสุด

ระดับความสำคัญ	ตัวดำเนินการ	คำอธิบาย	ทิศทางการจัดกลุ่ม (Associativity)
1 (สูงสุด)	`()` `[]` `{}`	วงเล็บครอบนิพจน์, การสร้างลิสต์, การสร้างดิกชันนารี/เซต	จากซ้ายไปขวา
2	`x[index]` `x[i:j]` `x.attr` `x(...)`	การเข้าถึงด้วยดัชนี, การสไลซ์, การอ้างถึงแอตทริบิวต์, การเรียกใช้ฟังก์ชัน/เมธอด	จากซ้ายไปขวา
3	`**`	การยกกำลัง (Exponentiation)	**จากขวาไปซ้าย**
4	`+x` `-x` `~x`	เครื่องหมายบวกเอกภาค, เครื่องหมายลบเอกภาค, บิตไวส์ NOT	จากขวาไปซ้าย
5	`*` `/` `//` `%`	การคูณ, การหารจริง, การหารปัดเศษลง, มอดุโล	จากซ้ายไปขวา
6	`+` `-`	การบวก, การลบ	จากซ้ายไปขวา
7	`<<` `>>`	บิตไวส์เลื่อนบิตไปทางซ้าย, เลื่อนบิตไปทางขวา	จากซ้ายไปขวา
8	`&`	บิตไวส์ AND	จากซ้ายไปขวา
9	`^`	บิตไวส์ XOR	จากซ้ายไปขวา
10	`|`	บิตไวส์ OR	จากซ้ายไปขวา
11	`==` `!=` `<` `<=` `>` `>=` `is` `is not` `in` `not in`	การเปรียบเทียบค่า, การตรวจสอบเอกลักษณ์, การตรวจสอบการเป็นสมาชิกในลำดับ	เชื่อมโยงต่อเนื่อง (Chained)
12	`not`	ตัวดำเนินการเชิงตรรกะ NOT	จากขวาไปซ้าย
13	`and`	ตัวดำเนินการเชิงตรรกะ AND (แบบลัดวงจร)	จากซ้ายไปขวา
14	`or`	ตัวดำเนินการเชิงตรรกะ OR (แบบลัดวงจร)	จากซ้ายไปขวา
15	`if - else`	นิพจน์เงื่อนไขแบบ Ternary Expression	จากขวาไปซ้าย
16	`lambda`	นิพจน์ฟังก์ชันนิรนาม Lambda	ไม่มี
17 (ต่ำสุด)	`:=`	Walrus Operator (การกำหนดค่าภายในนิพจน์ Python 3.8+)	จากขวาไปซ้าย

ภาคผนวก B

ตารางอ้างอิงฟังก์ชันบิลต์อินมาตรฐานของ Python

ไลบรารีมาตรฐานของภาษา Python มีฟังก์ชันบิลต์อินพื้นฐานที่สามารถเรียกใช้งานได้ทั่วโลกโดยไม่ต้องนำเข้า (Import) ใด ๆ:

ฟังก์ชันบิลต์อิน	คำอธิบายการทำงานหลัก	หมวดหมู่
`abs(x)`	คืนค่าสัมบูรณ์ของตัวเลข	คณิตศาสตร์
`all(iterable)`	คืนค่า `True` หากสมาชิกทุกตัวในคอลเลกชันเป็นจริง (Truthy)	ตรรกศาสตร์
`any(iterable)`	คืนค่า `True` หากมีสมาชิกอย่างน้อยหนึ่งตัวในคอลเลกชันเป็นจริง	ตรรกศาสตร์
`bin(x)` / `hex(x)`	แปลงจำนวนเต็มให้เป็นสตริงฐานสอง (`0b...`) หรือฐานสิบหก (`0x...`)	การแปลงข้อมูล
`bool(x)`	แปลงค่าเป็นบูลีนโดยผ่านการทดสอบ Truth Value Testing	ชนิดข้อมูล
`callable(obj)`	คืนค่า `True` หากวัตถุนั้นสามารถเรียกทำงานแบบฟังก์ชันได้	การสำรวจวัตถุ
`dict(**kw)`	สร้างโครงสร้างดิกชันนารีคู่คีย์-ค่าชุดใหม่	ชนิดข้อมูล
`dir([obj])`	คืนค่ารายชื่อแอตทริบิวต์และเมธอดทั้งหมดของวัตถุ	การสำรวจวัตถุ
`divmod(a, b)`	คืนค่าทูเพิลของ `(a // b, a % b)` ในคำสั่งเดียว	คณิตศาสตร์
`enumerate(iter)`	คืนค่าคู่ลำดับของ `(index, item)` จากคอลเลกชัน	การวนซ้ำ
`filter(fn, iter)`	สร้างอิตเทอเรเตอร์ที่กรองเฉพาะสมาชิกที่ฟังก์ชันประเมินว่าเป็นจริง	เชิงฟังก์ชัน
`float(x)`	แปลงสตริงหรือตัวเลขให้เป็นทศนิยมแบบ Float	ชนิดข้อมูล
`getattr(obj, name)`	ดึงค่าแอตทริบิวต์ตามชื่อที่ระบุออกมาแบบไดนามิก	การสำรวจวัตถุ
`isinstance(o, c)`	คืนค่า `True` หากวัตถุเป็นอินสแตนซ์ของคลาสหรือคลาสย่อยที่ระบุ	ชนิดข้อมูล
`len(s)`	คืนค่าความยาวหรือจำนวนสมาชิกในลำดับ/คอลเลกชัน	ลำดับข้อมูล
`list([iterable])`	สร้างอินสแตนซ์ของลิสต์ใหม่	ชนิดข้อมูล
`map(fn, iter)`	ประยุกต์ใช้ฟังก์ชันกับสมาชิกทุกตัวในคอลเลกชัน	เชิงฟังก์ชัน
`max(arg, ...)`	คืนค่าสมาชิกที่มีค่ามากที่สุด	คณิตศาสตร์
`min(arg, ...)`	คืนค่าสมาชิกที่มีค่าน้อยที่สุด	คณิตศาสตร์
`open(file, mode)`	เปิดสตรีมไฟล์และคืนค่าเป็นบริบทอ็อบเจกต์ไฟล์	I/O
`print(*objects)`	พิมพ์ข้อความของวัตถุออกสู่สตรีมมาตรฐาน (หน้าจอเทอร์มินัล)	I/O
`range(start, stop)`	ตัวสร้างลำดับตัวเลขทางคณิตศาสตร์แบบไม่เปลี่ยนแปลงรูป	การวนซ้ำ
`reversed(seq)`	คืนค่าอิตเทอเรเตอร์ที่ท่องข้อมูลย้อนหลังจากหลังมาหน้า	ลำดับข้อมูล
`round(n, ndigits)`	ปัดเศษตัวเลขตามจำนวนตำแหน่งทศนิยมที่กำหนด (ใช้การปัดเศษของนายธนาคาร)	คณิตศาสตร์
`set([iterable])`	สร้างคอลเลกชันเซตที่มีสมาชิกไม่ซ้ำกัน	ชนิดข้อมูล
`sorted(iterable)`	คืนค่าเป็นลิสต์ใหม่ที่จัดเรียงลำดับสมาชิกแล้ว	ลำดับข้อมูล
`str(obj)`	คืนค่าสตริงที่แสดงผลข้อความของวัตถุ	ชนิดข้อมูล

ฟังก์ชันบิลต์อิน	คำอธิบายการทำงานหลัก	หมวดหมู่
`sum(iterable)`	หาผลรวมของสมาชิกทั้งหมดในคอลเลกชันจากซ้ายไปขวา	คณิตศาสตร์
`tuple([iterable])`	สร้างอินสแตนซ์ของทูเพิลใหม่ที่ไม่เปลี่ยนแปลงรูป	ชนิดข้อมูล
`type(obj)`	คืนค่าคลาสชนิดข้อมูลที่แท้จริงของวัตถุ	การสำรวจวัตถุ
`zip(*iterables)`	ท่องผ่านหลายคอลเลกชันแบบขนานกันโดยส่งคืนทูเพิลคู่ลำดับ	การวนซ้ำ

ภาคผนวก C

สรุปการใช้งานเมธอดของสตริงและคอลเลกชัน

ตารางอ้างอิงเมธอดหลักของโครงสร้างข้อมูล

โครงสร้างข้อมูล	เมธอดและการดำเนินการหลัก	คุณสมบัติไม่แปรเปลี่ยน (Invariants)
`str`	`lower()`, `upper()`, `strip()`, `split()`, `join()`, `replace()`, `find()`, `startswith()`, `format()`	ลำดับข้อความยูนิโค้ดที่ไม่เปลี่ยนแปลงรูป (Immutable) ทุกการแก้ไขจะคืนสตริงใหม่เสมอ
`list`	`append()`, `extend()`, `insert()`, `remove()`, `pop()`, `clear()`, `sort()`, `reverse()`, `index()`	ลำดับข้อมูลแบบเปลี่ยนแปลงได้ (Mutable) จัดเรียงตามลำดับ การแก้ไขจะทำบนตัวลิสต์เดิมโดยตรง
`tuple`	`count(x)`, `index(x)`, การ Unpacking, การสไลซ์	ลำดับข้อมูลที่ไม่เปลี่ยนแปลงรูป (Immutable) สามารถนำไปแฮชได้หากสมาชิกภายในแฮชได้ทั้งหมด
`dict`	`keys()`, `values()`, `items()`, `get()`, `setdefault()`, `pop()`, `update()`, `clear()`	การจับคู่คีย์-ค่าแบบเชื่อมโยง ค้นหารวดเร็วระดับ $O(1)$ รักษาลำดับตามการแทรกข้อมูล
`set`	`add()`, `remove()`, `discard()`, `pop()`, `union()`, `intersection()`, `difference()`	คอลเลกชันสมาชิกไม่ซ้ำและไม่เรียงลำดับ ตรวจสอบการเป็นสมาชิกได้รวดเร็วระดับ $O(1)$